Memory-Efficient Versioned File Indexer

CS253 : Software Development and Operations

Assignment 1 Report

230875

Ronit Kumar

ronitk23@iitk.ac.in

March 2026

Contents

1 Introduction **2**

2 System Architecture **2**

2.1 Class Diagram 3

2.2 Class Descriptions 4

2.2.1 BufferedFileReader 4

2.2.2 Tokenizer 4

2.2.3 WordIndex 4

2.2.4 VersionedIndex 4

2.2.5 Query Hierarchy 4

3 Requirements Satisfaction **5**

3.1 Design Requirements 5

3.2 C++ Feature Requirements 5

3.2.1 Inheritance and Runtime Polymorphism 5

3.2.2 Function Overloading 5

3.2.3 Templates 5

3.2.4 Exception Handling 5

4 Additional Features **6**

4.1 Interactive Mode 6

4.2 Optional Progress Bar 6

5 Memory Constraint Verification **7**

5.1 Dataset Sizes 7

5.2 Memory Measurement 7

5.3 Results 7

6 Usage Examples **8**

6.1 Word Frequency Query 8

6.2 Top-K Query 8

6.3 Difference Query 8

6.4 Interactive Mode 8

7 Conclusion **9**

1 Introduction

Processing large text files efficiently is a common challenge in real-world systems such as log analyzers, search engines, and monitoring tools. Loading an entire file into memory often leads to excessive memory consumption and poor scalability. This project addresses these challenges by implementing a **memory-efficient versioned file indexer** that builds a word-level index over large text files using a **fixed-size buffer**, processing files incrementally and storing only the necessary indexing information.

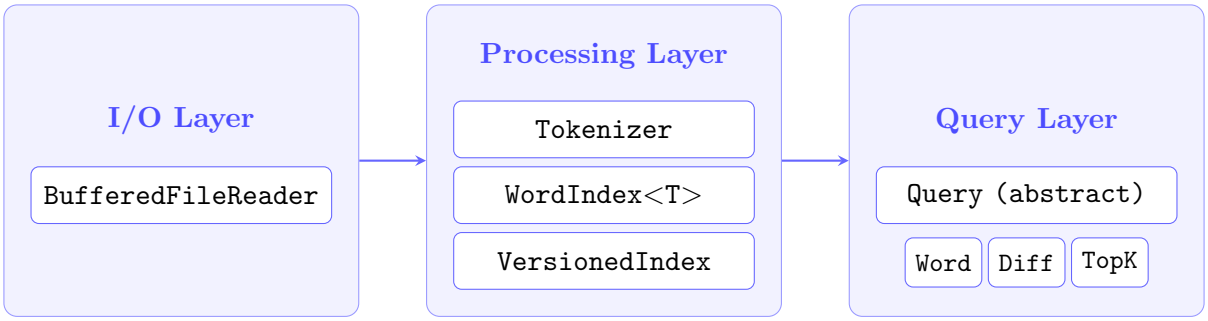
The system supports multiple file versions and provides three types of queries:

Query Type	Description
Word Count	Retrieve the frequency of a specific word in a given file version.
Difference	Compare word frequencies across two file versions.
Top-K	Find the k most frequent words in a version.

The complete source code, documentation, and a sample execution screenshot are available in the project repository:
C++ Source Code
README Documentation
Execution Screenshot
The following sections detail the system architecture, requirements satisfaction, memory constraint verification, and usage examples.

2 System Architecture

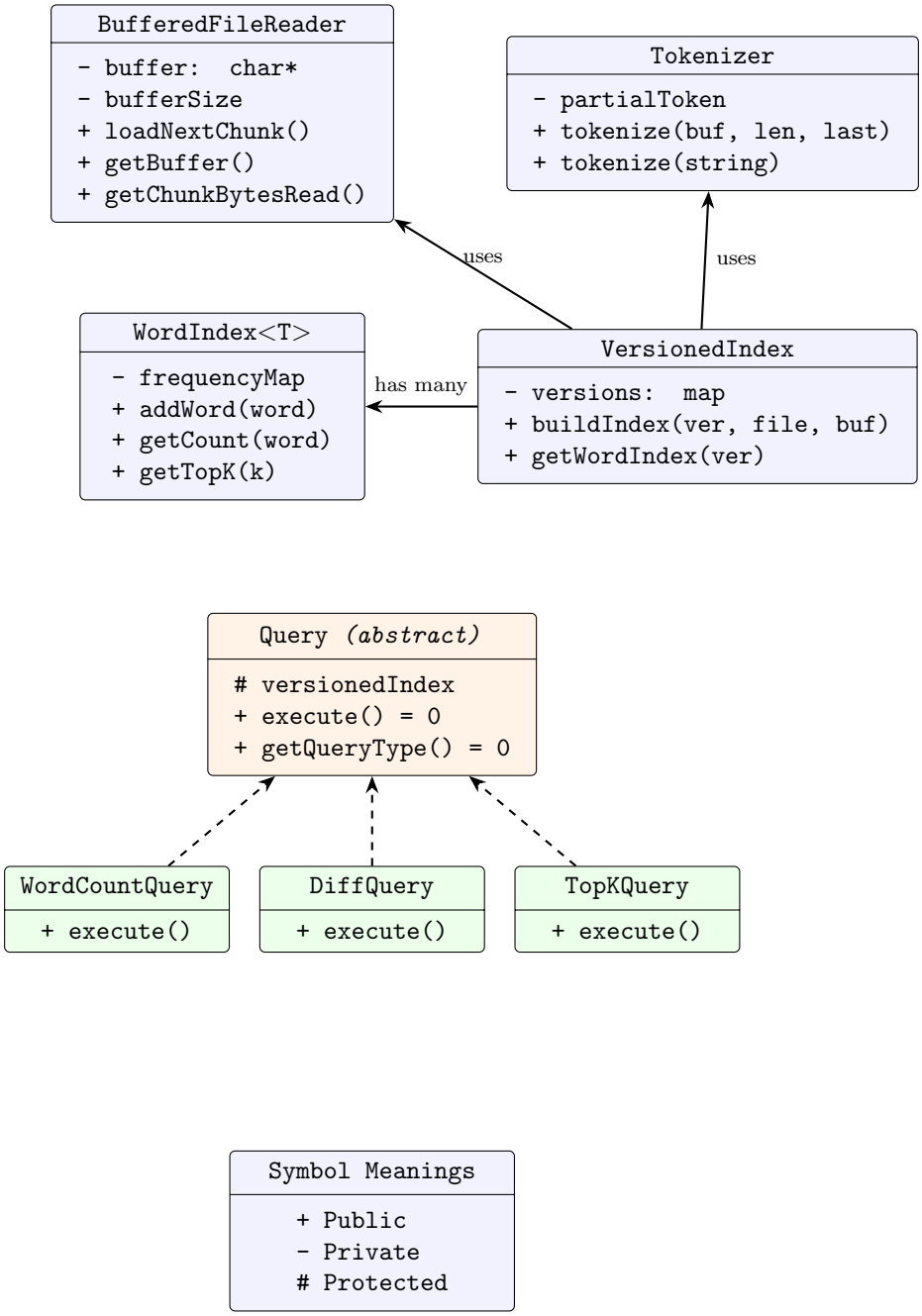
The program is composed of six principal classes, organized into three layers. Data flows from left to right through the pipeline:



The `BufferedFileReader` reads the file in fixed-size chunks, ensuring the entire file is never resident in memory. Each chunk is passed to the `Tokenizer`, which extracts words while correctly handling tokens that span chunk boundaries. The resulting tokens are fed into `WordIndex<T>`, which stores word frequencies, while `VersionedIndex` manages multiple version of frequency maps. Finally, queries are executed against the constructed index through the abstract `Query` interface, with `WordCountQuery`, `DiffQuery`, and `TopKQuery` providing solid implementations via polymorphic dispatch.

2.1 Class Diagram

The following diagram illustrates the relationships between the core classes:



2.2 Class Descriptions

2.2.1 BufferedFileReader

This class reads files incrementally using a fixed-size buffer.
How It Works :

1. A single buffer of size B KB (where $256 \leq B \leq 1024$) is allocated on the heap.
2. The file is read in chunks of B KB via `ifstream::read()`.
3. Each chunk is passed to the `Tokenizer` to extracts words and feed them to `WordIndex`.
4. After the chunk is processed, the same buffer is reused for the next chunk, ensuring that memory usage remains constant regardless of file size.

Copy operations are disabled to prevent double-free errors.

2.2.2 Tokenizer

The tokenizer extracts words from raw buffer data, words are defined as contiguous sequences of alphanumeric characters, it converts all characters to lowercase for case-insensitive indexing. It handles tokens split across buffer boundaries using a `partialToken` accumulator. For example, if one chunk ends with `erro` and the next begins with `r debug`, the tokenizer correctly reconstructs the word `error`.

2.2.3 WordIndex

A template class parameterized on the count type, defaulting to `long long` for large files but customizable to `int`, `unsigned`, etc. The template is instantiated as `WordIndex<long long>` in the rest of the program.
It stores word frequencies using an `unordered_map`. It provides `addWord()`, `getCount()`, and `getTopK()` operations, keeping the indexing logic generic and reusable.

2.2.4 VersionedIndex

Maintains separate `WordIndex` instances for each file version, allowing independent frequency tracking and cross-version comparison.

2.2.5 Query Hierarchy

The `Query` class defines a query interface with **pure virtual functions** (`execute()` and `getQueryType()`) for all query types. making it an abstract base class that cannot be instantiated directly.
Classes derived from `Query`:

1. `WordCountQuery` — looks up a single word's frequency
2. `DiffQuery` — computes frequency difference across two versions
3. `TopKQuery` — retrieves the top-K most frequent words

Runtime polymorphism enables queries to be dispatched through a base-class pointer.

3 Requirements Satisfaction

3.1 Design Requirements

The assignment requires at least four user-defined classes with clear, well-separated responsibilities. This implementation provides five core classes, each handling a distinct stage of the indexing pipeline:

1. **BufferedFileReader** — memory-bounded file I/O.
2. **Tokenizer** — word extraction and normalization.
3. **WordIndex<T>** — frequency storage and retrieval.
4. **VersionedIndex** — multi-version index management.
5. **Query hierarchy** — polymorphic query execution.

3.2 C++ Feature Requirements

3.2.1 Inheritance and Runtime Polymorphism

The abstract class `Query` defines a common interface for all query types. Derived classes, `WordCountQuery`, `DiffQuery`, and `TopKQuery`, inherit this interface and implement their own `execute()` methods. Queries are dispatched through base-class pointers, enabling dynamic dispatch.

3.2.2 Function Overloading

The `Tokenizer` class provides two overloaded `tokenize` methods:

1. `tokenize(const char*,size_t,bool)` : for buffer data with boundary handling.
2. `tokenize(const string&)`:a convenience overload for string inputs.

Overload 1 is the core tokenizer used during chunk-by-chunk file processing. **Overload 2** is a convenience wrapper used for normalizing query words (lowercasing). The compiler selects the correct overload based on the argument types at compile time.

3.2.3 Templates

The `WordIndex` class is implemented as a class template (`WordIndex<T>`), making the frequency-counting logic generic and reusable across different key types.

3.2.4 Exception Handling

Specific exceptions are thrown for:

1. `invalid_argument` — buffer size out of range, unknown query type, missing arguments, non-positive K
2. `runtime_error` — file not found, duplicate version name, version not found

4 Additional Features

4.1 Interactive Mode

If any required command-line argument is missing, the program **interactively prompts** the user instead of failing. The query type is asked first (since it determines which other arguments are needed), followed by query-specific parameters:

```
$ ./analyzer
Enter query type (word / diff / top): word
Enter buffer size in KB (256-1024): 256
Enter file path (--file): test_logs.txt
Enter version name (--version): v1
Enter word to search (--word): error
```

Listing 1: Interactive session example

Partial arguments are also supported, only the missing ones are prompted. The execution timer starts after all inputs are collected so that user think-time does not inflate the reported time.

4.2 Optional Progress Bar

An optional progress bar (in `progressbar.h`) is auto-detected at compile time using C++17's `__has_include`:

```
#if __has_include("progressbar.h")
    #include "progressbar.h"
    #define HAS_PROGRESSBAR 1
#else
    #define HAS_PROGRESSBAR 0
#endif
```

Listing 2: Conditional progress bar inclusion

If present, a progress bar is displayed on `stderr` during indexing (so it doesn't pollute query output on `stdout`). If absent, the program compiles and runs identically without it. All progress bar code is clearly boxed with comment markers for easy removal.on stderr during indexing (so it doesn't pollute query output on stdout). If absent, the program compiles and runs identically without it. All progress bar code is clearly boxed with comment markers for easy removal.

5 Memory Constraint Verification

A core design goal is to ensure that memory usage remains *independent* of file size. To verify this property, the base dataset `test_logs.txt` (~109 MB) was concatenated multiple times to produce progressively larger files, all containing the same set of unique words but differing vastly in total size.

5.1 Dataset Sizes

```
$ du -sh * | sort -hr
11G     100_test_logs.txt
5.3G    50_test_logs.txt
1.1G    10_test_logs.txt
541M    5_test_logs.txt
109M    test_logs.txt
```

*Files range from 109 MB to 11 GB — a **100**× increase in size.*

5.2 Memory Measurement

Memory usage was measured using the `/usr/bin/time -v` utility, which reports the *Maximum resident set size* of the process:

```
/usr/bin/time -v ./analyzer --file test_logs.txt --version v1
    --buffer 512 --query word --word error 2>&1 | grep "Maximum
    resident"
/usr/bin/time -v ./analyzer --file 100_test_logs.txt --version v1
    --buffer 512 --query word --word error 2>&1 | grep "Maximum
    resident"
```

5.3 Results

Memory usage remained nearly constant across all file sizes:

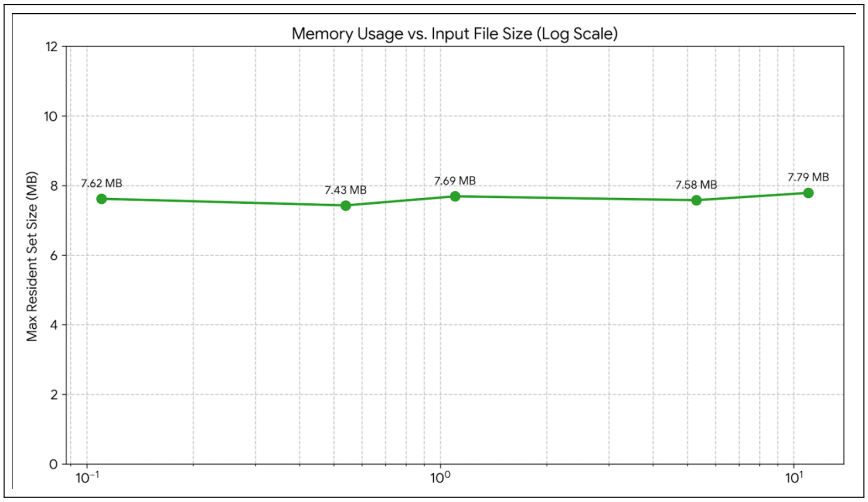


Figure 1: Memory usage vs. file size — remains flat as file size grows.

This confirms that memory consumption depends only on the number of *unique words* (which remains constant across concatenated files), not on the total file size.

6 Usage Examples

The analyzer supports both command-line and interactive modes. Below are examples of each query type, illustrating how the inheritance-based query hierarchy operates in practice.

6.1 Word Frequency Query

To look up how often a specific word appears in a file version:

```
$ ./analyzer --file test_logs.txt --version v1 --buffer 512 --query
    word --word error
```

6.2 Top-K Query

To retrieve the most frequent words in a version:

```
$ ./analyzer --file test_logs.txt --version v1 --buffer 512 --query
    top --top 10
```

6.3 Difference Query

To compare word frequencies between two file versions:

```
$ ./analyzer --file1 test_logs.txt --version1 v1 --file2
    verbose_logs.txt --version2 v2 --buffer 512 --query diff --word
    error
```

6.4 Interactive Mode

When no command-line arguments are provided, the program enters an interactive mode and prompts the user for each required parameter:

```
$ ./analyzer
Enter query type (word / diff / top): word
Enter buffer size in KB (256--1024): 512
Enter file path (--file): test_logs.txt
Enter version name (--version): v1
Enter word to search (--word): error
```

This makes the program accessible even to users unfamiliar with the command-line interface.

7 Conclusion

This project demonstrates the design and implementation of a scalable, memory-efficient file indexing system. Key achievements include:

1. Incremental file processing using fixed-size buffers, with verified constant memory usage across files ranging from 109 MB to 11 GB.
2. Efficient word-level indexing with support for multiple file versions.
3. A clean object-oriented architecture employing inheritance, runtime polymorphism, templates, function overloading, and exception handling.
4. Flexible query processing via a polymorphic dispatch mechanism.

The system successfully processes large datasets while maintaining stable memory usage and providing efficient, user-friendly query support.

Repository: https://github.com/Ronit-k/CS253_MemoryEfficientVersionedFileIndexer